Programmation procédurale (UAA11)

Atelier : les fonctions

Objectifs de l'atelier :

- ✔ Comprendre pourquoi on découpe un programme en fonctions.
- ✔ Écrire et appeler une fonction, avec ou sans paramètre.
- ✔ Découvrir le rôle du prototype.
- ✔ Distinguer une fonction qui affiche et une fonction qui renvoie une valeur.
- ✔ Observer la portée des variables et le passage par copie.

Avant de commencer

Cet atelier se fait seul, avant la séance de théorie. Tu n'as rien à apprendre par cœur : tu vas compiler de petits programmes, les exécuter, les modifier, et découvrir par toi-même comment le C se comporte. Les règles que tu formuleras serviront de point de départ à la séance.

Les programmes à télécharger :

https://lucasembrechts.github.io/UAA11/fichiers/atelier_fonctions.zip

Comment travailler :

- décompresse l'archive : tu obtiens un fichier `.c` par programme. Le nom du fichier est indiqué au-dessus de chaque programme ;
- ouvre chaque fichier dans ton éditeur habituel, compile-le et exécute-le ;
- **avant** chaque exécution, écris ta prédiction. Ne la corrige pas après coup : une prédiction fausse est justement ce qui va t'apprendre quelque chose ;
- certains programmes ne compilent pas : c'est voulu. Lis alors attentivement le message du compilateur, il fait partie de la découverte ;
- ne cherche pas la réponse dans le cours ou sur Internet : l'atelier est conçu pour que tu la trouves en observant ;
- à la fin de chaque section, résume ta découverte dans le cadre « Ma règle ». L'ensemble de tes règles formera ta fiche pour la séance théorique.

Section 0 : le problème

Fichier : section0.c

```
#include <stdio.h>

void main(void) {
    for (int i = 0; i < 10; i++) printf("*");
    printf("\n");
    printf("  MENU DU JOUR\n");
    for (int i = 0; i < 10; i++) printf("*");
    printf("\n");
    printf("  Soupe : 2.50\n");
    printf("  Plat  : 8.00\n");
    for (int i = 0; i < 10; i++) printf("*");
    printf("\n");
}
```

Prédis, puis exécute

Qu'affiche ce programme ?

Explique

On veut maintenant des lignes de 20 étoiles. Combien de modifications faut-il faire ? Et si le menu comptait dix rubriques séparées par des lignes ?

✎ Ma règle : quel est le défaut de ce programme ?

Section 1 : une première fonction

Fichier : section1.c

```
#include <stdio.h>

void afficherLigne(void) {
    for (int i = 0; i < 10; i++) printf("*");
    printf("\n");
}

void main(void) {
    afficherLigne();
    printf("  MENU DU JOUR\n");
    afficherLigne();
    printf("  Soupe : 2.50\n");
    printf("  Plat  : 8.00\n");
    afficherLigne();
}
```

Prédis, puis exécute

L'affichage est-il différent de celui de la section 0 ?

Explique

Combien de fois le code qui dessine une ligne est-il écrit ? Combien de fois est-il exécuté ?

Que se passe-t-il, dans l'ordre, quand le programme rencontre `afficherLigne();` ?

Modifie

Déplace toute la fonction `afficherLigne` **sous** la fonction `main`, puis compile. Que se passe-t-il ? Pourquoi, selon toi ?

Ajoute au-dessus de `main` la ligne `void afficherLigne(void);` (la première ligne de la fonction, suivie d'un point-virgule). Compile. Que fait cette ligne ?

> 📖 Cette ligne s'appelle le **prototype** de la fonction : elle annonce au compilateur le nom de la fonction, ce qu'elle reçoit entre les parenthèses et ce qu'elle renvoie.

Le fichier suivant contient une version de la fonction qui reçoit la longueur de la ligne.

`Fichier : section1_parametre.c`

```
#include <stdio.h>

void afficherLigne(int longueur);

void main(void) {
    afficherLigne(20);
    printf("  MENU DU JOUR\n");
    afficherLigne(20);
    printf("  Soupe : 2.50\n");
    printf("  Plat  : 8.00\n");
    afficherLigne(5);
}

void afficherLigne(int longueur) {
    for (int i = 0; i < longueur; i++) printf("*");
    printf("\n");
}
```

Prédis l'affichage, puis exécute. D'où la variable `longueur` reçoit-elle sa valeur ?

> 📖 La variable `longueur`, déclarée entre les parenthèses de la fonction, s'appelle un **paramètre**. Écrire `afficherLigne(20);`, c'est **appeler** la fonction : la valeur 20 est transmise au paramètre.

✎ Ma règle : qu'est-ce qu'une fonction ? À quoi servent le prototype et les paramètres ?

Section 2 : renvoyer une valeur

Fichier : section2.c

```
#include <stdio.h>

int carre(int x);

void main(void) {
    int a = carre(4);
    printf("%d\n", a);
    printf("%d\n", carre(3) + carre(2));
    carre(5);
    printf("%d\n", carre(carre(2)));
}

int carre(int x) {
    return x * x;
}
```

Prédis

Qu'affiche ce programme ?

Exécute, puis explique

Que devient le résultat de `carre(5)` (ligne 9) ?

Dans `carre(carre(2))`, dans quel ordre les calculs se font-ils ?

Que signifie le `int` devant `carre` dans le prototype ?

Modifie

`Fichier : section2_void.c`

```
#include <stdio.h>

void afficherCarre(int x);

void main(void) {
    int a = afficherCarre(4);
    printf("%d\n", a);
}

void afficherCarre(int x) {
    printf("%d\n", x * x);
}
```

Cette version **affiche** le carré au lieu de le **renvoyer**. Compile-t-elle ? Quelle différence fais-tu entre afficher un résultat et le renvoyer ?

✎ Ma règle : à quoi sert return ?

Section 3 : chaque fonction a ses variables

Fichier : section3.c

```
#include <stdio.h>

void f(void);

void main(void) {
    int compteur = 5;
    f();
    printf("main : compteur = %d\n", compteur);
}

void f(void) {
    int compteur = 100;
    printf("f : compteur = %d\n", compteur);
}
```

Prédis, puis exécute

Qu'affiche ce programme ?

Explique

Les deux variables `compteur` sont-elles la même variable ?

Modifie

Supprime la ligne `int compteur = 100;` dans `f`, puis compile. Que se passe-t-il ? Pourquoi `f` ne voit-elle pas le `compteur` de `main` ?

> 📖 Une variable déclarée dans une fonction (ou dans une boucle) est une variable **locale** : elle n'existe que dans ce bloc.

Fichier : section3_boucle.c

```
#include <stdio.h>
```

```
void main(void) {
    int i = 50;
    for (int i = 0; i < 3; i++) {
        printf("%d ", i);
    }
    printf("%d\n", i);
}
```

Prédis l'affichage de ce programme, puis vérifie.

✎ Ma règle : où une variable existe-t-elle ?

Section 4 : la fonction reçoit une copie

Fichier : section4.c

```c
#include <stdio.h>

void doubler(int n);

void main(void) {
    int prix = 10;
    doubler(prix);
    printf("prix = %d\n", prix);
}

void doubler(int n) {
    n = n * 2;
    printf("dans doubler : n = %d\n", n);
}
```

Prédis, puis exécute

Qu'affiche ce programme ?

Explique

La fonction a bien doublé `n`. Pourquoi `prix` n'a-t-il pas changé ?

> 📖 On dit que `prix` est **passé par copie** : la fonction reçoit une copie de sa valeur, jamais la variable elle-même.

Modifie

Renomme le paramètre `n` en `prix` dans la fonction (prototype compris). Le résultat change-t-il ?

Modifie le programme pour que `prix` vaille vraiment 20 à la fin (indice : la section 2).

✎ Ma règle : une fonction peut-elle modifier une variable qu'on lui passe ?

Section 5 : une fonction qui répond oui ou non

Fichier : section5.c

```c
#include <stdio.h>
#include <stdbool.h>

bool estPair(int n);

void main(void) {
    for (int i = 1; i <= 6; i++) {
        if (estPair(i)) {
            printf("%d est pair\n", i);
        }
    }
    printf("%d\n", estPair(7));
}

bool estPair(int n) {
    if (n % 2 == 0) {
        return true;
    } else {
        return false;
    }
}
```

Prédis, puis exécute

Qu'affiche ce programme ?

Explique

Pourquoi le dernier affichage est-il 0 ?

Dans `if (estPair(i))`, qu'est-ce que le `if` teste exactement ?

Modifie

Remplace tout le corps de `estPair` par la seule ligne `return n % 2 == 0;`. Le résultat change-t-il ? Pourquoi ?

✎ Ma règle : comment écrire une fonction qui répond par oui ou non ?

Défi final

Le fichier `defi.c` calcule le prix d'entrée à la piscine, mais tout est écrit dans `main`. Découpe-le en trois fonctions, sans changer son comportement :

Fichier : defi.c

```
#include <stdio.h>

void main(void) {
    int age;
    printf("Age : ");
    scanf("%d", &age);
    getchar();
    while (age < 0 || age > 120) {
        printf("Age invalide. Age : ");
        scanf("%d", &age);
        getchar();
    }
    float prix;
    if (age < 12) {
        prix = 2.50;
    } else if (age < 65) {
        prix = 4.00;
    } else {
        prix = 3.00;
    }
    printf("----------------\n");
    printf("Entree piscine\n");
    printf("Age  : %d ans\n", age);
    printf("Prix : %.2f euros\n", prix);
    printf("----------------\n");
}
```

- `obtenirAge` demande l'âge jusqu'à ce qu'il soit valide (entre 0 et 120) et le renvoie ;
- `calculerPrix` reçoit l'âge et renvoie le prix ;
- `afficherTicket` reçoit l'âge et le prix et affiche le ticket ;
- à la fin, `main` ne contient plus que trois appels.

L'exécution doit rester identique :

```
Age : -3
Age invalide. Age : 70
----------------
Entree piscine
Age  : 70 ans
Prix : 3.00 euros
----------------
```

Pour chacune de tes fonctions, indique ses entrées et sa sortie.